Observation Task 1: Java Programming Fundamentals

Variables, Data Types, Control Structures, and Methods

1. Objective

To understand and implement the basic programming constructs of Java, including variables, data types, operators, conditional statements, loops, and methods via a "Student Utility Program".

2. Step-by-Step Process & Algorithm

- **Start** program execution and display a menu using a `do-while` loop.
- **Part A (Java Basics):** Use `Scanner` to read Name, Roll Number, and Marks. Calculate `Total` and `Percentage = (Marks/500)*100`. Display formatted details.
- **Part B (Conditionals):**
  - Use `if-else` or ternary operators to check Even/Odd and Largest of Three Numbers.
  - Use a `switch` statement to map an integer (1-7) to a Day of the Week.
- **Part C (Loops):**
  - Use a `for` loop to iterate from 1 to N to calculate the sum.
  - Use a `for` loop to print the Fibonacci series up to a given number of terms.
- **Part D (Methods):** Call parameterized static methods `findFactorial(n)`, `isPrime(n)`, `findMax(a,b)`, and `calculateCircleArea(r)` from `main()` and print their return values.
- **Stop** execution when the user selects the Exit option.

3. Source Code

```java
import java.util.Scanner;

public class StudentUtilityProgram {

    // Part D: Methods
    public static long findFactorial(int n) {
        long fact = 1;
        for (int i = 1; i <= n; i++) fact *= i;
        return fact;
    }

    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static int findMax(int a, int b) {
        return (a > b) ? a : b;
    }

    public static double calculateCircleArea(double radius) {
        return Math.PI * radius * radius;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n--- Student Utility Program ---");
            System.out.println("1. Part A: Java Basics (Student Info)");
            System.out.println("2. Part B: Conditional Statements");
            System.out.println("3. Part C: Looping Statements");
            System.out.println("4. Part D: Methods Implementation");
```

```
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    // Part A
                    System.out.print("Enter Name: ");
                    String name = sc.next();
                    System.out.print("Enter Roll Number: ");
                    int roll = sc.nextInt();
                    System.out.print("Enter Marks (out of 500): ");
                    double marks = sc.nextDouble();
                    double percentage = (marks / 500) * 100;
                    System.out.println("--- Student Result ---");
                    System.out.println("Name: " + name + " | Roll: " + roll);
                    System.out.println("Total Marks: " + marks + " | Percentage: " + percentage + "%");
                    break;
                case 2:
                    // Part B
                    System.out.print("Enter a number to check Even/Odd: ");
                    int num = sc.nextInt();
                    System.out.println(num + " is " + (num % 2 == 0 ? "Even" : "Odd"));

                    System.out.print("Enter three numbers to find largest: ");
                    int n1 = sc.nextInt(), n2 = sc.nextInt(), n3 = sc.nextInt();
                    int largest = (n1 > n2) ? (n1 > n3 ? n1 : n3) : (n2 > n3 ? n2 : n3);
                    System.out.println("Largest is: " + largest);

                    System.out.print("Enter day number (1-7): ");
                    int day = sc.nextInt();
                    switch (day) {
                        case 1: System.out.println("Monday"); break;
                        case 2: System.out.println("Tuesday"); break;
                        case 3: System.out.println("Wednesday"); break;
                        case 4: System.out.println("Thursday"); break;
                        case 5: System.out.println("Friday"); break;
                        case 6: System.out.println("Saturday"); break;
                        case 7: System.out.println("Sunday"); break;
                        default: System.out.println("Invalid day");
                    }
                    break;
                case 3:
                    // Part C
                    System.out.print("Enter N to find sum of first N natural numbers: ");
                    int n = sc.nextInt();
                    int sum = 0;
                    for (int i = 1; i <= n; i++) sum += i;
                    System.out.println("Sum is: " + sum);

                    System.out.print("Enter terms for Fibonacci: ");
                    int terms = sc.nextInt();
                    int a = 0, b = 1;
                    System.out.print("Fibonacci Series: " + a + " " + b);
                    for (int i = 2; i < terms; i++) {
                        int next = a + b;
                        System.out.print(" " + next);
                        a = b; b = next;
                    }
                    System.out.println();
                    break;
                case 4:
                    // Part D
                    System.out.println("Factorial of 5: " + findFactorial(5));
                    System.out.println("Is 7 Prime? " + isPrime(7));
                    System.out.println("Max of 10 and 20: " + findMax(10, 20));
                    System.out.println("Area of circle (r=5): " + calculateCircleArea(5.0));
                    break;
                case 5:
                    System.out.println("Exiting...");
                    break;
                default:
                    System.out.println("Invalid choice!");
            }
        } while (choice != 5);
        sc.close();
```

```
    }
}
```

4. Observation Questions & Answers

Q: What is the structure of a Java program?

A: A standard Java program consists of a package declaration (optional), import statements, class declaration(s), and the `main` method (`public static void main(String[] args)`) which serves as the entry point. Variables and methods are defined within the class block.

Q: Differentiate between primitive and non-primitive data types.

A: Primitive data types (e.g., int, float, char, boolean) store simple values directly in memory and have a predefined size. Non-primitive data types (e.g., Strings, Arrays, Classes) are reference types that point to objects dynamically allocated in the heap memory.

Q: Compare if-else and switch statements.

A: `if-else` is used for evaluating complex boolean expressions and ranges. `switch` is preferred for executing specific blocks of code based on a single variable's exact match against multiple discrete constants (cases), making the code cleaner for menu-driven logic.

Q: Explain the differences between for, while, and do-while loops.

A: `for` is an entry-controlled loop used when the number of iterations is known. `while` is an entry-controlled loop used when the condition must be checked before execution. `do-while` is an exit-controlled loop that guarantees execution at least once before checking the condition.

Q: What are methods? Why are they useful in Java?

A: Methods are blocks of code designed to perform a specific task. They are useful for code reusability, modularity, and readability, allowing developers to avoid duplicating code.

Q: Explain the difference between formal parameters and actual parameters.

A: Formal parameters are the variables defined in the method signature (declaration) to receive data. Actual parameters (arguments) are the concrete values passed to the method during its invocation.

Observation Task 2: Object-Oriented Programming in Java

Classes, Objects, Encapsulation, Inheritance, Polymorphism & Abstraction

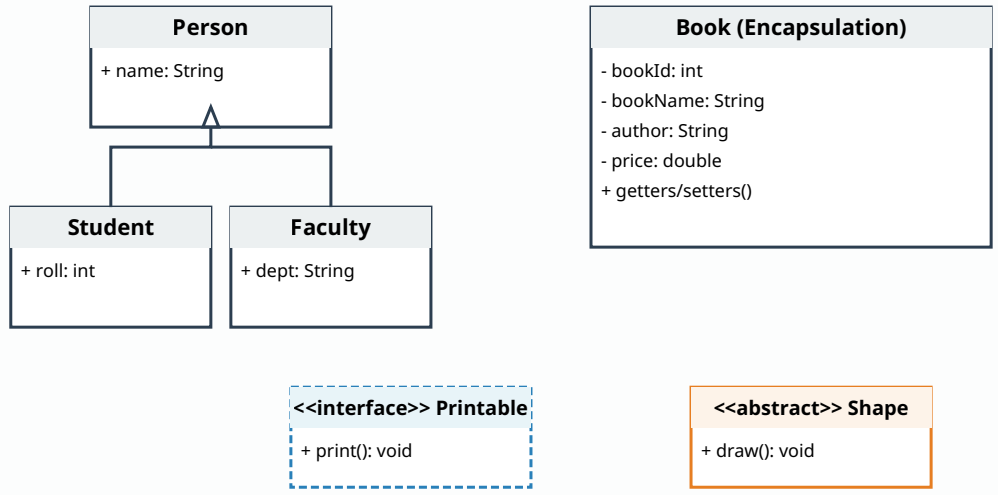
1. Objective

To understand and implement the core Object-Oriented Programming (OOP) concepts in Java by developing a "Library Management System".

2. Step-by-Step Process & Algorithm

- **Part A & B (Encapsulation & Constructors):** Create a `Book` class with private fields (bookId, bookName, author, price). Provide default and parameterized constructors. Implement public getters/setters for controlled access.
- **Part C (Inheritance):** Create a base class `Person` with a `name` property. Derive `Student` and `Faculty` classes from `Person`, utilizing the `super()` keyword to initialize inherited properties.
- **Part D (Polymorphism):**
  - **Compile-time (Overloading):** Create an `AreaCalculator` class with multiple `area()` methods having different parameter signatures.
  - **Run-time (Overriding):** Create a `Vehicle` base class and override its `display()` method in derived `Car` and `Bike` classes.
- **Part E (Abstraction & Interfaces):** Create an abstract class `Shape` with an abstract method `draw()`, implemented by `Circle` and `Rectangle`. Create an interface `Printable` with a `print()` method, implemented by a `Report` class.

3. UML Class Diagrams (Conceptual)



4. Source Code

```java
// Part A & B: Classes, Objects, Constructors, Encapsulation
class Book {
    private int bookId;
    private String bookName;
    private String author;
    private double price;

    public Book() {
        this.bookId = 0;
        this.bookName = "Unknown";
        this.author = "Unknown";
        this.price = 0.0;
    }
```

```
    public Book(int bookId, String bookName, String author, double price) {
        this.bookId = bookId;
        this.bookName = bookName;
        this.author = author;
        this.price = price;
    }

    public int getBookId() { return bookId; }
    public String getBookName() { return bookName; }
    public String getAuthor() { return author; }
    public double getPrice() { return price; }
}

// Part C: Inheritance
class Person {
    String name;
    public Person(String name) { this.name = name; }
    void displayPerson() { System.out.println("Name: " + name); }
}

class Student extends Person {
    int roll;
    public Student(String name, int roll) { super(name); this.roll = roll; }
    void displayStudent() { displayPerson(); System.out.println("Roll: " + roll); }
}

class Faculty extends Person {
    String department;
    public Faculty(String name, String department) { super(name); this.department = department; }
    void displayFaculty() { displayPerson(); System.out.println("Dept: " + department); }
}

// Part D: Polymorphism (Overloading & Overriding)
class AreaCalculator {
    public double area(double radius) { return Math.PI * radius * radius; }
    public double area(double length, double breadth) { return length * breadth; }
}

class Vehicle {
    void display() { System.out.println("This is a generic Vehicle."); }
}

class Car extends Vehicle {
    @Override
    void display() { System.out.println("This is a Car."); }
}

class Bike extends Vehicle {
    @Override
    void display() { System.out.println("This is a Bike."); }
}

// Part E: Abstraction and Interfaces
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() { System.out.println("Drawing a Circle."); }
}

class Rectangle extends Shape {
    void draw() { System.out.println("Drawing a Rectangle."); }
}

interface Printable {
    void print();
}

class Report implements Printable {
    public void print() { System.out.println("Printing the Report..."); }
}

public class LibraryManagementSystem {
    public static void main(String[] args) {
        System.out.println("--- OOP Concepts Demonstration ---");
```

```
        // Encapsulation & Constructors
        Book b1 = new Book(101, "Java Programming", "James Gosling", 500.0);
        System.out.println("Book: " + b1.getBookName() + " by " + b1.getAuthor());

        // Inheritance
        Student s = new Student("Alice", 1);
        s.displayStudent();

        // Polymorphism
        AreaCalculator calc = new AreaCalculator();
        System.out.println("Area of circle (r=5): " + calc.area(5));

        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();
        v1.display();
        v2.display();

        // Abstraction & Interfaces
        Shape shape = new Circle();
        shape.draw();

        Printable doc = new Report();
        doc.print();
    }
}
```

5. Observation Questions & Answers

Q: What is Object-Oriented Programming?

A: OOP is a programming paradigm organized around "objects" (which contain data in the form of fields and code in the form of procedures/methods) rather than "actions" and logic. It focuses on modularity, reusability, and simulating real-world entities.

Q: Differentiate between a Class and an Object.

A: A **Class** is a blueprint or template that defines the structure, properties, and behaviors (methods) of a specific type. An **Object** is a concrete, runtime instance of a class occupying memory space.

Q: Explain the purpose of constructors.

A: Constructors are special methods invoked automatically when an object is instantiated. Their primary purpose is to initialize the newly created object's state (its variables) before it is used.

Q: What is Encapsulation? How is it implemented in Java?

A: Encapsulation is the mechanism of hiding data from the outside world to protect it from unauthorized access. In Java, it is implemented by declaring class variables as `private` and providing public getter and setter methods to access and modify them safely.

Q: Differentiate between Method Overloading and Method Overriding.

A: **Overloading** (Compile-time polymorphism) occurs when multiple methods in the same class share the same name but have different parameter lists. **Overriding** (Runtime polymorphism) occurs when a subclass provides a specific implementation for a method already defined in its parent class (same name and signature).

Q: Explain the four pillars of Object-Oriented Programming.

A: 1. **Encapsulation:** Bundling data and methods into a single unit (e.g., a Class with private fields). 2. **Inheritance:** Acquiring properties of a parent class to promote reusability (e.g., `Student extends Person`). 3. **Polymorphism:** The ability of an object to take many forms (e.g., Method Overloading/Overriding). 4. **Abstraction:** Hiding internal implementation details and showing only functional operations (e.g., Abstract classes and Interfaces).

Q: Differentiate between Abstract Classes and Interfaces.

> A: An **Abstract Class** can contain both abstract methods and concrete implementations, and a class can extend only one abstract class. An **Interface** is a pure contract containing only abstract methods (prior to Java 8) and constants, and a single class can implement multiple interfaces, allowing multiple inheritance of type.